Data Warehouse, Data Lake và Data Lakehouse

(Data Architecture)

Minh-Tuan Le **Thi-Vang Nguyen**

Proofreading:

Mục lục

I. Giới thiệu **3**

II. Ba lớp của một nền tảng phân tích **4**

III. Data Warehouse **5**

III.1. OLTP và OLAP: hai kiểu workload khác nhau 5

III.2. Lưu trữ theo dòng và theo cột 5

III.3. Schema-on-write 7

III.4. Mô hình hóa: Inmon, Kimball và Star Schema 7

III.5. Ưu điểm và giới hạn 8

IV. Data Lake **8**

IV.1. Object storage 8

IV.2. Định dạng file 9

IV.3. Schema-on-read và partition kiểu Hive 9

IV.4. Khi không ai sở hữu bảng: data swamp 10

V. Kiến trúc hai tầng và sự ra đời của Lakehouse **11**

VI. Bên trong Data Lakehouse **12**

VI.1. Table format là gì 12

VI.2. Cây metadata của Apache Iceberg 12

VI.3. Commit nguyên tử và ACID 13

VI.4. Schema evolution và partition evolution 14

VI.5. Thay đổi ở mức dòng: Copy-on-Write và Merge-on-Read . . . 15
VI.6. Time travel, tag và rollback . . . 15
VI.7. Delta Lake và Apache Hudi . . . 16
VI.8. Catalog: lớp điều khiển của Lakehouse . . . 17
VI.9. Bảo trì bảng (Day-2 operations) . . . 17

VII. So sánh và kết hợp ba kiến trúc . . . 19
VII.1. Bảng so sánh tổng hợp . . . 19
VII.2. Kiến trúc lai trong thực tế . . . 19
VII.3. Medallion và mô hình hóa dữ liệu . . . 20
VII.4. Khi nào chọn kiến trúc nào . . . 21

VIII. Thực hành: NYC Yellow Taxi từ Bronze đến Gold . . . 21
VIII.1. Môi trường và dữ liệu . . . 21
VIII.2. Silver: chuẩn hóa schema và nạp vào Iceberg . . . 22
VIII.3. Gold: Star Schema với DuckDB . . . 23
VIII.4. Hướng mở rộng . . . 24

IX. Lộ trình học tiếp . . . 24

X. Tổng kết . . . 25

Bảng thuật ngữ . . . 25

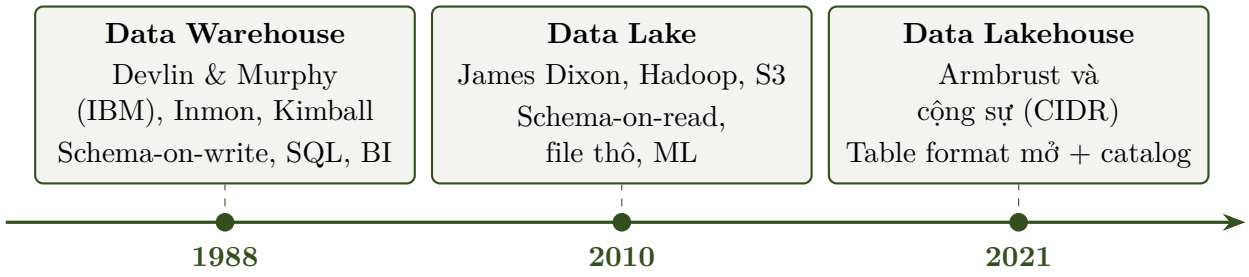
XI. Tài liệu tham khảo . . . 27

I. Giới thiệu

Taxi vàng ở New York thực hiện khoảng ba triệu chuyến xe mỗi tháng. Mỗi chuyến được ghi lại thời điểm đón và trả khách, quãng đường, giá cước, tiền tip và hình thức thanh toán [1]. Ba nhóm người cùng cần dữ liệu này:

- **Nhà phân tích BI** cần dashboard doanh thu theo ngày và theo hãng, cập nhật mỗi sáng.
- **Data scientist** cần toàn bộ lịch sử ở dạng thô để huấn luyện mô hình dự đoán tiền tip hoặc nhu cầu đặt xe.
- **Data engineer** cần một nơi lưu trữ không bị hỏng khi pipeline gặp lỗi, cho phép sửa dòng sai và truy lại trạng thái cũ.

Trong gần bốn mươi năm, câu hỏi "dữ liệu phân tích nên nằm ở đâu" đã có ba câu trả lời nối tiếp nhau (Hình 1). **Data Warehouse** được Devlin và Murphy mô tả năm 1988 [2], sau đó được Inmon và Kimball phát triển thành phương pháp luận [3, 4]. **Data Lake** do James Dixon đặt tên năm 2010 [5], gắn với Hadoop và sau này là cloud object storage. **Data Lakehouse** được Armbrust và cộng sự định nghĩa tại hội nghị CIDR năm 2021 [6].



Hình 1: Ba thế hệ kiến trúc dữ liệu phân tích.

Bài đọc giải thích ba kiến trúc từ cơ chế bên trong, không dừng ở bảng so sánh tính năng. Sau khi đọc, bạn có thể:

1. Phân biệt Data Warehouse, Data Lake và Data Lakehouse bằng một câu hỏi: lớp nào sở hữu khái niệm "bảng".
2. Giải thích vì sao Data Lake thuần dễ biến thành data swamp, và cách một table format như Apache Iceberg khắc phục bằng metadata cùng commit nguyên tử.
3. Thiết kế một nền tảng kết hợp cả ba kiến trúc theo mô hình Medallion.
4. Tự chạy pipeline Bronze → Silver → Gold trên laptop với dữ liệu NYC Taxi thật, không cần tài khoản cloud.

Lưu ý

Bài đọc giả định bạn đã biết SQL cơ bản (`SELECT`, `JOIN`, `GROUP BY`) và Python cơ bản. Mọi output trong bài được lấy từ việc chạy thật bộ code đi kèm (xem mục VIII.).

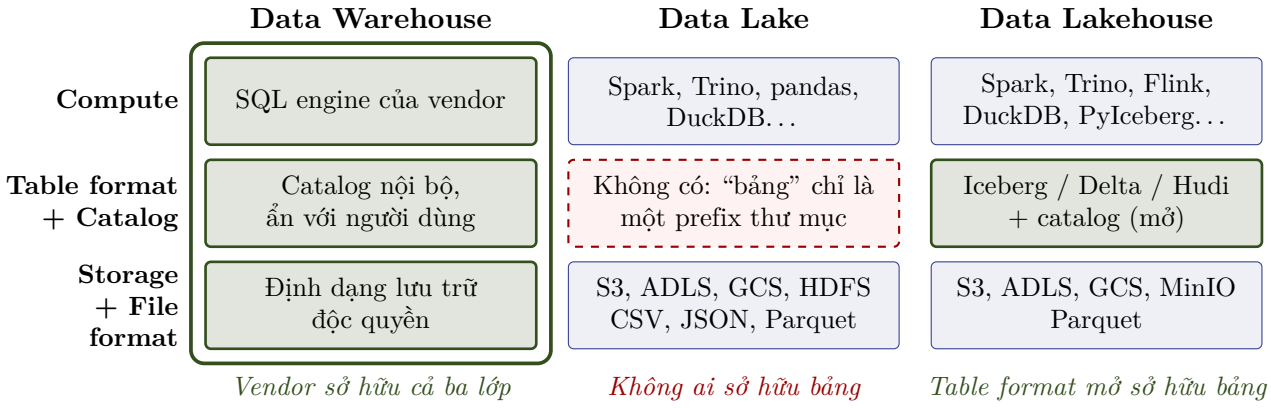
II. Ba lớp của một nền tảng phân tích

Mọi nền tảng phân tích đều gồm ba lớp:

- **Compute**: engine thực thi truy vấn SQL, biến đổi dữ liệu hoặc huấn luyện mô hình, ví dụ Spark, Trino, DuckDB.
- **Table format và catalog**: metadata cho biết một bảng gồm những file nào, schema ra sao, partition thế nào và đang ở phiên bản nào.
- **Storage và file format**: nơi các byte thực sự nằm (đĩa của database, HDFS, Amazon S3) cùng định dạng file (Parquet, CSV, JSON).

Khác biệt cốt lõi giữa ba kiến trúc nằm ở việc **ai sở hữu lớp ở giữa** (Hình 2):

- **Data Warehouse**: vendor sở hữu cả ba lớp. Người dùng chỉ làm việc qua SQL, còn file vật lý bị ẩn sau storage engine riêng. Snowflake, BigQuery hay Redshift tính tiền compute và storage riêng, nhưng engine bên ngoài vẫn phải đi qua giao diện của vendor để đọc dữ liệu.
- **Data Lake**: không ai sở hữu lớp bảng. "Bảng" chỉ là một quy ước, tức tập file nằm dưới một prefix như `s3://bucket/orders/`. Mỗi engine tự liệt kê file và tự suy ra schema.
- **Data Lakehouse**: một table format mở cùng một catalog sở hữu định nghĩa bảng, còn dữ liệu vẫn là file Parquet trên object storage. Engine hỏi catalog để biết snapshot hiện hành gồm những file nào, rồi đọc hoặc ghi trực tiếp các file đó.



Hình 2: Ba lớp của nền tảng phân tích và lớp sở hữu "bảng" trong từng kiến trúc. Khung xanh lá là lớp sở hữu bảng, khung xanh dương là thành phần mở mà nhiều bên dùng chung.

Câu hỏi để phân loại một hệ thống

Khi một engine muốn đọc bảng `orders`, nó hỏi ai để biết bảng gồm những file nào?

- Hỏi engine của vendor ⇒ Data Warehouse.

- Tự liệt kê thư mục ⇒ Data Lake.
- Hỏi catalog của một table format mở ⇒ Data Lakehouse.

III. Data Warehouse

III.1. OLTP và OLAP: hai kiểu workload khác nhau

Database phía sau ứng dụng như PostgreSQL hay MySQL được tối ưu cho OLTP: rất nhiều giao dịch nhỏ, mỗi giao dịch chạm vài dòng. Câu hỏi phân tích lại có dạng OLAP: quét hàng triệu dòng nhưng chỉ cần vài cột, rồi tổng hợp theo nhiều chiều (Bảng 1).

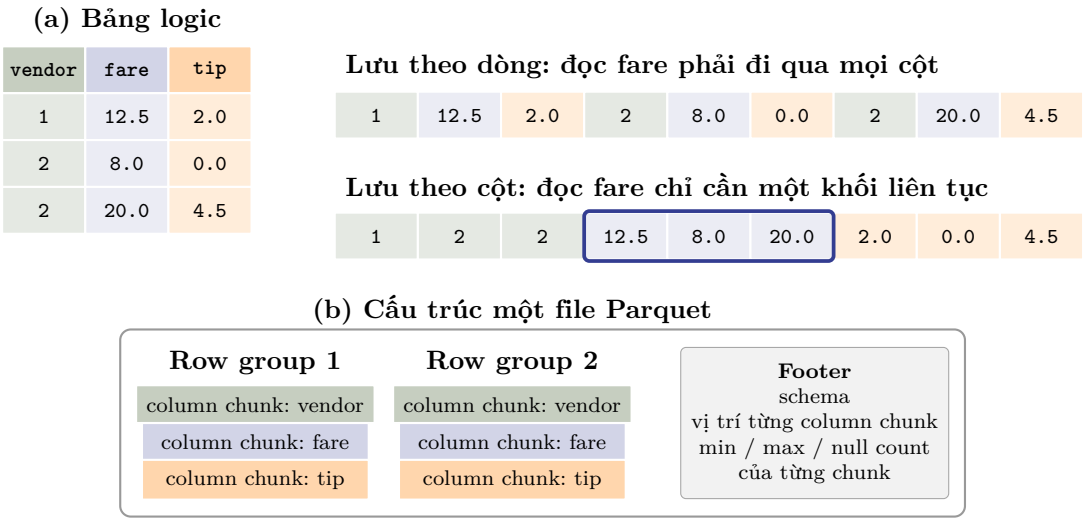
Tiêu chí	**OLTP**	**OLAP**
Truy vấn ví dụ	Cập nhật trạng thái chuyến xe #123	Doanh thu theo ngày của từng hãng trong hai tháng
Số dòng mỗi truy vấn	Vài dòng	Hàng triệu dòng
Số cột cần đọc	Gần như mọi cột của một dòng	Vài cột trên rất nhiều dòng
Thao tác chính	`INSERT`, `UPDATE`, `DELETE`	`SELECT` với `JOIN`, `GROUP BY`
Mô hình dữ liệu	Chuẩn hóa (3NF) để tránh trùng lặp	Star schema, chấp nhận phi chuẩn hóa
Cách lưu phù hợp	Theo dòng (row-oriented)	Theo cột (columnar)

Bảng 1: So sánh workload OLTP và OLAP.

Chạy truy vấn phân tích trực tiếp trên database vận hành gặp bốn vấn đề: truy vấn quét lớn tranh tài nguyên với giao dịch của khách hàng, schema chuẩn hóa buộc phải `JOIN` nhiều bảng, database chỉ giữ trạng thái hiện tại thay vì lịch sử, và dữ liệu nằm rải rác ở nhiều hệ thống. Data Warehouse là một hệ thống riêng cho phân tích, gom dữ liệu từ các nguồn về một nơi và lưu theo cách tối ưu cho OLAP.

III.2. Lưu trữ theo dòng và theo cột

Lưu theo dòng đặt liền nhau các giá trị của cùng một dòng; lưu theo cột đặt liền nhau các giá trị của cùng một cột (Hình 3a). Với truy vấn `SELECT avg(fare_amount) FROM trips`, cách lưu theo cột chỉ cần đọc một cột. Các giá trị cùng kiểu đứng cạnh nhau nên cũng nén tốt hơn.



Hình 3: (a) Cùng một bảng khi lưu theo dòng và theo cột. (b) File Parquet gồm các row group, mỗi row group chứa một column chunk cho từng cột, footer ở cuối file giữ schema và thống kê.

Apache Parquet là định dạng columnar mở phổ biến nhất [7]. Như Hình 3b, một file Parquet được chia thành các row group, mỗi row group chứa một column chunk cho từng cột, còn footer ở cuối file ghi schema cùng thống kê min, max, số giá trị null của từng column chunk. Engine đọc footer trước để thực hiện hai tối ưu:

- **Projection pushdown**: chỉ đọc column chunk của các cột mà truy vấn cần.
- **Predicate pushdown**: bỏ qua row group mà khoảng min–max cho thấy không thể thỏa điều kiện `WHERE`.

Đoạn code sau đọc footer của file chuyến xe tháng 1/2023 mà không cần đọc dữ liệu:

01_bronze_data_lake.py: đọc footer Parquet

```
pf = pq.ParquetFile(bronze_file("2023-01"))
meta = pf.metadata
print(f"Số dòng: {meta.num_rows:,} | Số cột: {meta.num_columns} | Số row group: {meta.
                                        num_row_groups}")
rg = meta.row_group(0)
for name in ["tpep_pickup_datetime", "fare_amount", "trip_distance"]:
    col = rg.column(pf.schema_arrow.get_field_index(name))
    st = col.statistics
    print(f"  {name:<22} {col.total_compressed_size / 1e6:6.1f} MB  min={st.min}  max={
                                        st.max}")
```

Output

```
Số dòng: 3,066,766 | Số cột: 19 | Số row group: 1
  tpep_pickup_datetime     11.4 MB  min=2008-12-31 23:01:42  max=2023-02-01 00:56:53
  fare_amount               3.1 MB  min=-900.0  max=1160.1
```

```
  trip_distance             4.3 MB  min=-0.0  max=258928.15
```

Chỉ riêng phần thống kê đã cho thấy dữ liệu có vấn đề: file tháng 1/2023 chứa chuyến đón khách năm 2008, giá cước âm và quãng đường hơn 258.000 dặm. Data Warehouse ngăn những dòng như vậy ngay từ lúc nạp, nhờ cơ chế dưới đây.

III.3. Schema-on-write

Data Warehouse áp dụng schema-on-write: bảng phải được khai báo trước bằng `CREATE TABLE` với kiểu dữ liệu và ràng buộc. Khi nạp, dòng không khớp kiểu bị từ chối. Cái giá là phải thiết kế schema trước khi có dữ liệu. Đổi lại, dữ liệu trong kho luôn đúng kiểu và người truy vấn không phải đoán cấu trúc.

III.4. Mô hình hóa: Inmon, Kimball và Star Schema

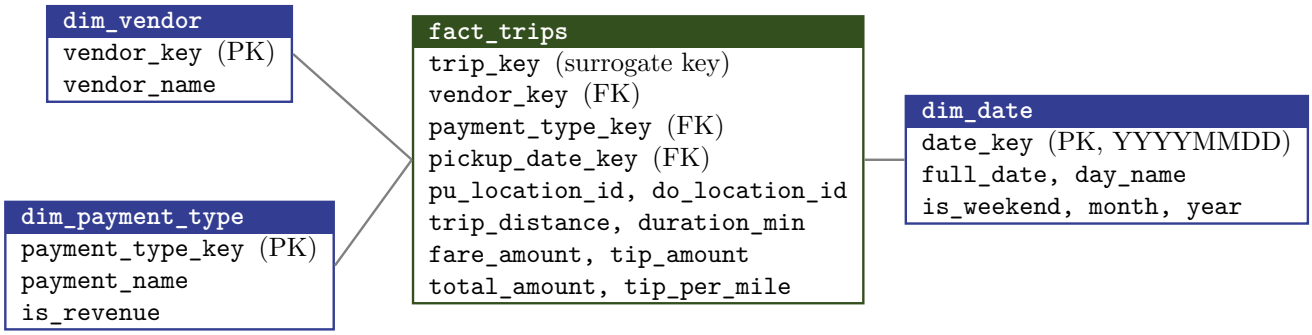
Có hai trường phái kinh điển để tổ chức dữ liệu trong warehouse:

- **Inmon** [3] xây một enterprise data warehouse chuẩn hóa theo 3NF làm nguồn sự thật chung, sau đó tách ra các data mart cho từng phòng ban.
- **Kimball** [4] xây trực tiếp theo từng quy trình nghiệp vụ bằng dimensional modeling (star schema). Các mart dùng chung những dimension đã thống nhất (conformed dimension).

Ngoài ra, **Data Vault 2.0** [8] tách dữ liệu thành hub (khóa nghiệp vụ), link (quan hệ) và satellite (thuộc tính kèm lịch sử). Cách này phù hợp khi tích hợp nhiều nguồn và cần lưu vết đầy đủ.

Star schema của Kimball gồm **fact table** chứa các phép đo của sự kiện nghiệp vụ và các **dimension table** chứa ngữ cảnh để lọc và nhóm. Kimball đề xuất bốn bước thiết kế: chọn quy trình nghiệp vụ, khai báo grain, xác định dimension, rồi xác định fact. Áp dụng cho dữ liệu taxi:

1. **Quy trình nghiệp vụ**: chuyến xe hoàn thành.
2. **Grain**: một dòng trong fact table là một chuyến xe.
3. **Dimension**: hãng vận hành (vendor), hình thức thanh toán, ngày đón khách.
4. **Fact**: quãng đường, thời lượng, giá cước, tiền tip, tổng tiền.



Hình 4: Star schema của phần thực hành: `fact_trips` ở giữa, nối tới ba dimension qua khóa ngoại.

Khóa của dimension nên là surrogate key do warehouse sinh ra thay vì dùng thẳng mã của nguồn. Ví dụ thật: data dictionary năm 2018 của NYC TLC ghi mã vendor 2 là *VeriFone Inc.*, còn bản năm 2025 ghi *Curb Mobility, LLC* [9]. Nếu báo cáo lịch sử cần hiển thị đúng tên tại thời điểm chuyến xe diễn ra, `dim_vendor` phải lưu theo SCD Type 2 (Bảng 2). Khi đó mã nguồn 2 ứng với hai dòng, và chỉ surrogate key mới phân biệt được chúng.

vendor_key	vendor_id	vendor_name	valid_from	valid_to	is_current
11	2	VeriFone Inc.	2009-01-01	2023-06-30	FALSE
27	2	Curb Mobility, LLC	2023-07-01	9999-12-31	TRUE

Bảng 2: `dim_vendor` theo SCD Type 2. Ngày hiệu lực chỉ để minh họa cơ chế, không phải ngày đổi tên thật.

III.5. Ưu điểm và giới hạn

Data Warehouse mạnh ở truy vấn SQL hiệu năng cao, phục vụ nhiều người dùng đồng thời, có ACID, phân quyền chi tiết và được vendor vận hành tự động. Giới hạn của nó đến từ chính việc vendor sở hữu cả ba lớp:

- Chi phí lưu trữ và compute tăng nhanh khi dữ liệu lên tới hàng trăm terabyte.
- Chủ yếu phục vụ dữ liệu có cấu trúc; log, ảnh, âm thanh khó đưa vào.
- Dữ liệu bị khóa trong định dạng riêng. Thư viện ML như pandas hay PyTorch phải xuất dữ liệu qua JDBC/ODBC, rất chậm khi dữ liệu lớn [6].

IV. Data Lake

IV.1. Object storage

James Dixon dùng hình ảnh "hồ dữ liệu" để mô tả cách lưu dữ liệu nguyên dạng và chỉ quyết định cách dùng về sau [5]. Ngày nay Data Lake chủ yếu nằm trên object storage: Amazon S3,

Google Cloud Storage, Azure Data Lake Storage, hoặc MinIO khi tự vận hành. Năm đặc điểm của object storage quyết định cách Data Lake hoạt động:

- Object storage là kho key–value. Key có dạng `bronze/yellow_tripdata/year=2023/month=01/part-0.parquet`, value là nội dung file. Không có thư mục thật; "thư mục" chỉ là phần prefix chung của các key.
- Liệt kê file theo prefix (LIST) là một lời gọi qua mạng và chậm dần khi prefix chứa hàng trăm nghìn file.
- Mỗi object được ghi trọn vẹn hoặc không được ghi, nhưng không có thao tác nguyên tử cho nhiều file cùng lúc.
- Chi phí lưu trữ thấp, và storage tách rời compute nên nhiều engine có thể dùng chung một bản dữ liệu.
- Từ tháng 12/2020, Amazon S3 đảm bảo strong read-after-write consistency: ghi xong thì mọi lần đọc sau đều thấy bản mới [10]. Đây là một điều kiện kỹ thuật để Lakehouse hoạt động đúng.

IV.2. Định dạng file

Data Lake chấp nhận mọi định dạng. Bảng 3 tóm tắt các định dạng phổ biến.

Định dạng	Cách lưu	Schema	Thường dùng cho
CSV	Dòng, văn bản	Không có kiểu dữ liệu	Trao đổi dữ liệu đơn giản
JSON / JSON Lines	Dòng, văn bản	Mỗi bản ghi tự mô tả, cho phép lồng nhau	Log ứng dụng, dữ liệu API
Avro	Dòng, nhị phân	Schema lưu trong file	Streaming, Kafka, metadata của Iceberg
Parquet	Cột, nhị phân	Schema và thống kê trong footer	Phân tích, Lakehouse
ORC	Cột, nhị phân	Schema và thống kê theo stripe	Hệ sinh thái Hive

Bảng 3: Các định dạng file thường gặp trong Data Lake.

IV.3. Schema-on-read và partition kiểu Hive

Data Lake theo nguyên tắc schema-on-read: ghi trước, hiểu sau. Không ai kiểm tra schema lúc ghi, còn engine suy ra schema lúc đọc. Để không phải quét toàn bộ dữ liệu, file được chia vào thư mục theo quy ước của Hive, tên thư mục có dạng `cột=giá_trị`. Khi gặp điều kiện `WHERE month = '01'`, engine chỉ liệt kê prefix tương ứng.

01_bronze_data_lake.py: "bảng" Data Lake chỉ là một glob

```
GLOB = f"{BRONZE_DIR}/**/*.parquet"
con = duckdb.connect()
con.sql(f"""
    SELECT year, month, count(*) AS trips, round(avg(fare_amount), 2) AS avg_fare
    FROM read_parquet('{GLOB}', hive_partitioning = true)
    GROUP BY ALL ORDER BY ALL
""").show()
```

Output

```
+-------+---------+---------+----------+
| year  |  month  |  trips  | avg_fare |
+-------+---------+---------+----------+
|  2023 | 01      | 3066766 |    18.37 |
|  2023 | 02      | 2913955 |    18.22 |
+-------+---------+---------+----------+
```

Partition kiểu Hive có ba hạn chế. Người dùng phải biết cột partition và lọc đúng trên cột đó: điều kiện trên `tpep_pickup_datetime` không tự loại được thư mục `year=/month=`. Đổi cách chia partition đồng nghĩa với ghi lại toàn bộ dữ liệu. Chia quá mịn sinh ra rất nhiều file nhỏ (small files problem), làm chậm cả thao tác LIST lẫn việc mở file.

IV.4. Khi không ai sở hữu bảng: data swamp

Script `01_bronze_data_lake.py` tái hiện bốn lỗi điển hình của Data Lake thuần. Lỗi thứ nhất có sẵn trong dữ liệu gốc: NYC TLC đổi schema giữa tháng 1 và tháng 2 năm 2023.

Lỗi 1: schema drift có thật giữa hai file nguồn

```
  VendorID              int64    ->  VendorID              int32
  passenger_count       double   ->  passenger_count       int64
  RatecodeID            double   ->  RatecodeID            int64
  store_and_fwd_flag    string   ->  store_and_fwd_flag    large_string
  PULocationID          int64    ->  PULocationID          int32
  DOLocationID          int64    ->  DOLocationID          int32
  airport_fee           double   ->  Airport_fee           double
```

Sáu cột đổi kiểu và một cột đổi tên chỉ bằng cách viết hoa chữ cái đầu. Engine đọc glob phải tự đoán cách hợp nhất, và mỗi engine có thể đoán một kiểu. Lỗi thứ hai xảy ra khi một job khác ghi file sai kiểu vào cùng prefix:

Lỗi 2: một file sai kiểu làm hỏng mọi truy vấn trên prefix

```
Truy vấn thất bại (ConversionException): failed to cast column "VendorID" from type
VARCHAR to BIGINT: : Could not convert string 'INVALID_ID' to INT64
```

Lỗi thứ ba là ghi dở dang. Một job ghi dữ liệu tháng 2 thành bốn file nhưng bị dừng sau file thứ hai:

Lỗi 3: partial write

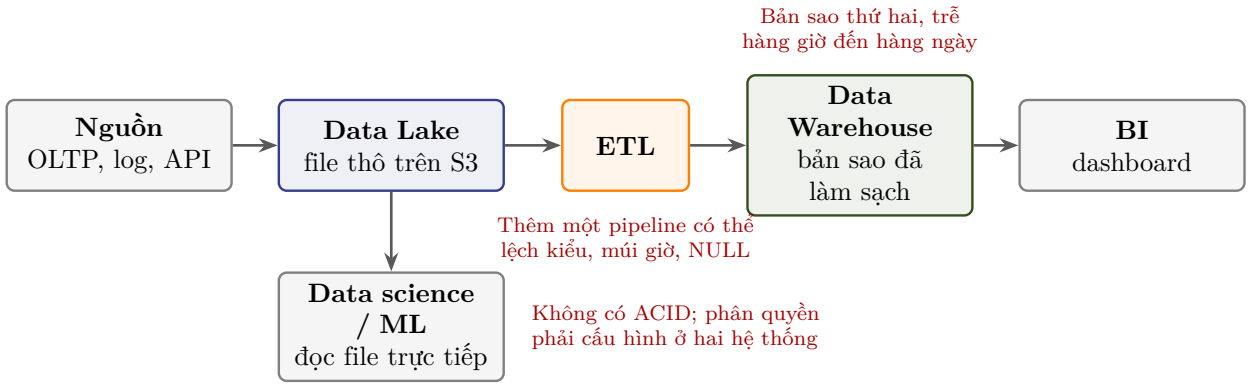
```
  đã ghi part-0.parquet
  đã ghi part-1.parquet
  !!! job bị kill khi đang ghi part-2
Reader thấy 1,456,978 dòng / 2,913,955 dòng (50%), không có cảnh báo nào.
```

Lỗi thứ tư là không có `UPDATE` hay `DELETE`. Muốn xóa 26 chuyến có giá cước trên 500 USD, ta phải đọc rồi ghi lại toàn bộ file 48 MB chứa hơn 3 triệu dòng. Trong lúc thay file, một reader khác có thể thấy file cũ, file mới hoặc không thấy file nào.

Gộp các lỗi trên với việc không có catalog (không biết ai ghi file nào, khi nào, file nào còn hợp lệ), Data Lake dần trở thành data swamp.

V. Kiến trúc hai tầng và sự ra đời của Lakehouse

Trong những năm 2010, cách làm phổ biến là kết hợp cả hai: Data Lake lưu dữ liệu thô và phục vụ ML, còn một phần dữ liệu được ETL tiếp sang Data Warehouse cho BI (Hình 5).



Hình 5: Kiến trúc hai tầng: Data Lake cho dữ liệu thô và ML, Data Warehouse cho BI.

Armbrust và cộng sự chỉ ra bốn vấn đề của kiến trúc này [6]:

1. **Độ tin cậy**: hai hệ thống phải được giữ khớp nhau, và mỗi bước ETL là một chỗ có thể làm lệch kiểu dữ liệu, múi giờ hay cách xử lý NULL.
2. **Dữ liệu cũ**: dữ liệu trong warehouse thường trễ hơn nguồn hàng giờ đến hàng ngày.
3. **Hỗ trợ ML hạn chế**: thư viện ML cần đọc file trực tiếp, trong khi warehouse chỉ mở cổng SQL; còn dữ liệu trên lake lại thiếu ACID và schema.
4. **Tổng chi phí sở hữu**: lưu hai bản dữ liệu, trả tiền compute cho ETL, và quản trị phân quyền ở hai nơi.

Ý tưởng Lakehouse là giữ dữ liệu trên object storage ở định dạng mở, rồi thêm một **lớp metadata có giao dịch** lên trên để có các tính năng quản trị của warehouse: ACID, quản lý phiên bản, tối ưu truy vấn [6]. Ba điều kiện đã cùng chín muồi vào khoảng năm 2020: định dạng columnar mở như Parquet, các table format như Delta Lake [11], Iceberg và Hudi, cùng strong consistency của object storage.

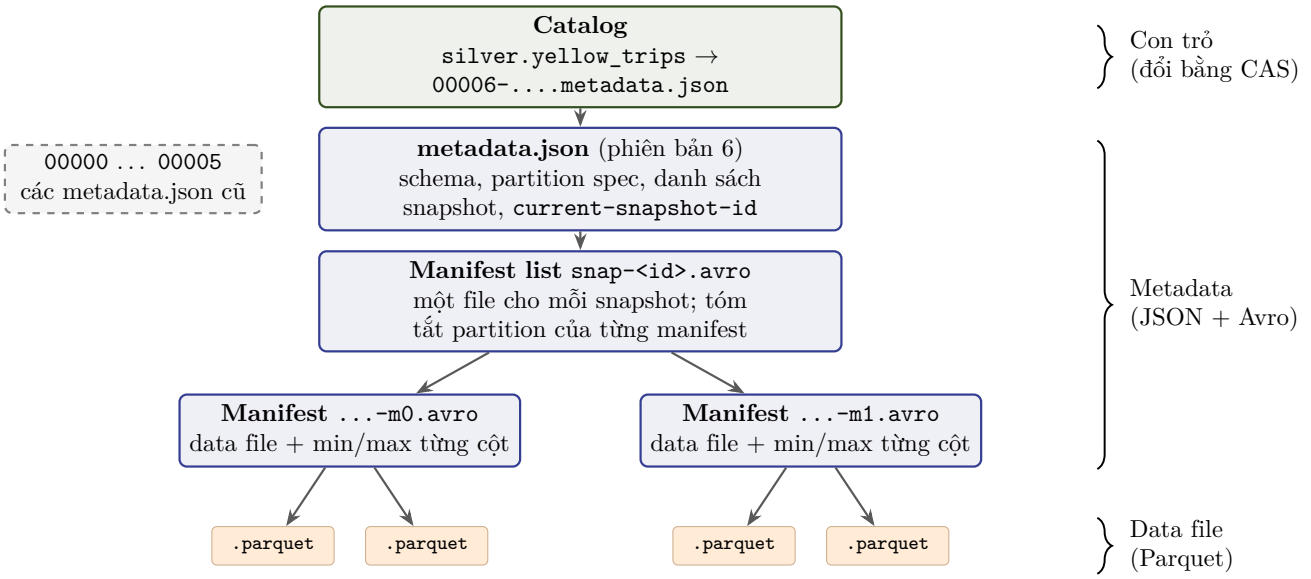
VI. Bên trong Data Lakehouse

VI.1. Table format là gì

Open table format là một đặc tả quy định cách dùng file metadata để mô tả một bảng: bảng gồm những data file nào, schema và partition ra sao, và có những phiên bản nào. Data file vẫn là Parquet và **bất biến**: không bao giờ bị sửa tại chỗ. Mỗi thay đổi tạo ra file mới cùng metadata mới. Ba table format phổ biến là Apache Iceberg (khởi đầu tại Netflix), Delta Lake (Databricks) và Apache Hudi (Uber). Phần này lấy Iceberg làm ví dụ chính vì phần thực hành dùng PyIceberg.

VI.2. Cây metadata của Apache Iceberg

Theo đặc tả Iceberg [12], metadata của một bảng là một cây năm tầng (Hình 6):



Hình 6: Cây metadata của một bảng Iceberg. Engine đi từ catalog xuống data file mà không cần liệt kê thư mục.

1. **Catalog** giữ một con trỏ duy nhất cho mỗi bảng: đường dẫn tới file `metadata.json` hiện hành.

2. **metadata.json** chứa schema (kèm các schema cũ), partition spec, thuộc tính bảng, danh sách snapshot và `current-snapshot-id`. Mỗi thay đổi tạo một `metadata.json` mới, đánh số `00000`, `00001`, ...

3. **Manifest list** (`snap-<id>-....avro`): mỗi snapshot có một manifest list, liệt kê các manifest của snapshot đó cùng khoảng giá trị partition của từng manifest.

4. **Manifest** (`...-m0.avro`): liệt kê data file hoặc delete file, kèm giá trị partition, số dòng, min, max và số null của từng cột.

5. **Data file**: file Parquet chứa dữ liệu thật.

Khi lập kế hoạch truy vấn, engine dùng phần tóm tắt partition trong manifest list để bỏ qua cả một manifest, rồi dùng min/max trong manifest để bỏ qua từng data file. Toàn bộ quá trình không cần lệnh LIST nào trên object storage. Sau lần append đầu tiên trong phần thực hành, thư mục `metadata/` có đúng bốn file:

02_silver_lakehouse.py: thư mục metadata sau commit đầu tiên

```
00000-d6d4d355-...-4058619005ac.metadata.json      1.6 KB   # tạo bảng rỗng
71240245-...-7edbede595e8-m0.avro                  5.4 KB   # manifest
snap-1178427703463806229-0-71240245-....avro       1.8 KB   # manifest list
00001-4fef47b5-...-609e32c717f7.metadata.json      2.6 KB   # sau khi append
```

VI.3. Commit nguyên tử và ACID

Toàn bộ bảo đảm ACID của Iceberg dựa trên một thao tác duy nhất: đổi con trỏ trong catalog bằng compare-and-swap (CAS).

Quy trình một lần ghi vào bảng Iceberg

1. Writer đọc metadata hiện hành, giả sử là phiên bản N.
2. Writer ghi các data file Parquet mới. Chưa reader nào nhìn thấy chúng vì chưa có metadata nào trỏ tới.
3. Writer ghi manifest, manifest list và `metadata.json` phiên bản $N+1$.
4. Writer yêu cầu catalog đổi con trỏ từ N sang $N+1$, **với điều kiện** con trỏ vẫn đang là N.
5. Nếu một writer khác đã commit trước, điều kiện ở bước 4 sai và commit thất bại. Writer đọc metadata mới, kiểm tra xung đột với thay đổi của người kia, tạo lại metadata rồi thử lại.

Từ quy trình này có đủ bốn tính chất ACID:

- **Atomicity**: bước 4 là một phép đổi con trỏ duy nhất. Job chết trước bước 4 chỉ để lại vài file không ai tham chiếu, bảng không bị ảnh hưởng. Đây chính là lời giải cho lỗi partial write đã gặp ở Data Lake (mục IV.).
- **Consistency**: writer phải ghi đúng schema của bảng. Ở phần thực hành, batch sai kiểu bị từ chối trước khi kịp commit.

- **Isolation**: reader cố định một snapshot khi bắt đầu đọc (snapshot isolation), nên không bao giờ thấy dữ liệu đang ghi dở.
- **Durability**: cả metadata lẫn data đều nằm trên storage bền vững.

Cơ chế này là **optimistic concurrency**: không khóa trước, xung đột chỉ được phát hiện lúc commit. Nó phù hợp với workload phân tích, nơi hiếm khi có nhiều writer cùng sửa một phần dữ liệu.

VI.4. Schema evolution và partition evolution

Iceberg định danh cột bằng **field ID** thay vì tên hay vị trí. Thêm, đổi tên, xóa hay đổi thứ tự cột vì thế chỉ là thay đổi metadata. File cũ vẫn được đọc đúng qua field ID, và cột mới trả về NULL cho các file ghi trước khi có cột.

Iceberg cũng hỗ trợ hidden partitioning: partition spec khai báo một transform trên cột nguồn, ví dụ `day(pickup_at)`, `month(pickup_at)`, `bucket(16, id)`. Người dùng chỉ viết điều kiện trên cột gốc, còn engine tự quy đổi sang partition. Khi đổi partition spec (partition evolution), chỉ dữ liệu ghi sau mới theo spec mới; file cũ giữ spec cũ, và engine lập kế hoạch riêng cho từng spec.

02_silver_lakehouse.py: hai thay đổi chỉ chạm vào metadata

```
with tbl.update_schema() as update:
    update.add_column("tip_per_mile", DoubleType(), doc="tip_amount / trip_distance")
with tbl.update_spec() as spec:
    spec.add_field("pickup_at", DayTransform(), "pickup_day")
```

Output

```
Schema hiện tại có 16 cột, cột mới có field_id = 16
Partition spec hiện tại: [
  1000: pickup_day: day(2)
]
Snapshot hiện tại vẫn là 1178427703463806229: chưa file dữ liệu nào bị ghi lại.
```

Sau đó dữ liệu tháng 2 được append theo spec mới. Bảng giờ có file thuộc hai spec, và truy vấn một ngày chỉ phải đọc đúng một file:

Output: file theo từng partition spec

```
         so_file  so_dong
spec_id
0              1  2998337     # tháng 01, không partition
1             28  2850309     # tháng 02, partition theo ngày
Truy vấn ngày 15/02 chỉ cần đọc 1/29 file nhờ partition pruning.
```

VI.5. Thay đổi ở mức dòng: Copy-on-Write và Merge-on-Read

Data file là bất biến, vậy `UPDATE` và `DELETE` hoạt động thế nào? Table format dùng một trong hai chiến lược (Bảng 4).

Tiêu chí	Copy-on-Write (CoW)	Merge-on-Read (MoR)
Cách làm	Ghi lại toàn bộ file chứa dòng bị đổi	Ghi riêng thông tin dòng bị xóa; `UPDATE` = xóa + chèn
Chi phí ghi	Cao: sửa vài dòng vẫn phải ghi lại cả file	Thấp: chỉ ghi phần thay đổi
Chi phí đọc	Thấp: đọc thẳng data file	Cao hơn: phải gộp delete vào dữ liệu lúc đọc
Phù hợp	Bảng đọc nhiều, cập nhật theo lô	CDC, streaming upsert, cập nhật thường xuyên; cần compaction định kỳ

Bảng 4: Hai chiến lược thay đổi dữ liệu ở mức dòng.

Với MoR, Iceberg format version 2 có hai loại delete file: *position delete* ghi đường dẫn file và vị trí dòng bị xóa, *equality delete* ghi điều kiện theo giá trị cột. Format version 3 bổ sung deletion vector, mỗi data file có một bitmap đánh dấu dòng đã xóa, lưu trong file Puffin [12]. Iceberg 1.11.0 (phát hành ngày 19/5/2026) được xem là phiên bản mà các tính năng v3 như deletion vector và kiểu `VARIANT` đã sẵn sàng cho môi trường production [13, 14].

PyIceberg thực hiện `DELETE` theo kiểu CoW, nên snapshot sinh ra có operation là `overwrite`:

02_silver_lakehouse.py: DELETE và lịch sử snapshot

```
Số dòng: 5,848,646 -> 5,848,609 (xóa 37 dòng)
  22:37:59  snapshot=1178427703463806229  parent=None                  append
  22:38:00  snapshot=5737701426491917244  parent=1178427703463806229  append
  22:38:02  snapshot=5667705168284501283  parent=5737701426491917244  overwrite
```

VI.6. Time travel, tag và rollback

Snapshot cũ vẫn nằm trong metadata cho tới khi bị expire, nên có thể đọc bảng tại bất kỳ snapshot nào (time travel). **Tag** đặt tên cho một snapshot quan trọng, chẳng hạn trạng thái chốt sổ cuối tháng. **Branch** cho phép ghi thử nghiệm trên một nhánh tách biệt. **Rollback** đưa con trỏ về snapshot cũ khi pipeline ghi sai.

02_silver_lakehouse.py: time travel

```
tbl.manage_snapshots().create_tag(snap_jan, "after_jan_load").commit()
as_of_jan = tbl.scan(snapshot_id=snap_jan).count()
as_of_feb = tbl.scan(snapshot_id=snap_feb).count()
```

Output

```
Snapshot 1 (tag after_jan_load): 2,998,337 dòng
Snapshot 2 (trước khi DELETE)  : 5,848,646 dòng
Hiện tại                       : 5,848,609 dòng
tip_per_mile của các dòng tháng 01 (ghi trước khi có cột): [None, None, None]
```

VI.7. Delta Lake và Apache Hudi

Delta Lake lưu trạng thái bảng trong thư mục `_delta_log/`, gồm các file JSON đánh số tuần tự (`00000000000000000000.json`, `...01.json`, ...). Mỗi commit là một file JSON ghi các action thêm hoặc bỏ data file. Định kỳ, mặc định cứ 10 commit, log được gộp thành một checkpoint Parquet để reader không phải đọc hàng nghìn file JSON [11, 15]. Commit thành công khi writer tạo được file JSON mang số kế tiếp; nếu hai writer cùng tạo một số thì chỉ một bên thắng. Delta 4.1.0 (tháng 3/2026) bổ sung, ở dạng preview, bảng do catalog quản lý và server-side planning qua Iceberg REST API [16].

Apache Hudi tổ chức metadata quanh một **timeline** ghi lại các action (commit, delta commit, compaction, clean) theo thời gian, có hai loại bảng CoW và MoR, và có index ở mức bản ghi giúp tìm nhanh file chứa một khóa cần upsert [17]. Hudi phù hợp nhất với CDC và streaming upsert tần suất cao.

Tiêu chí	Apache Iceberg	Delta Lake	Apache Hudi
Nguồn gốc	Netflix; Apache Software Foundation	Databricks; Linux Foundation	Uber; Apache Software Foundation
Metadata	Cây: metadata.json → manifest list → manifest	Log JSON tuần tự + checkpoint Parquet	Timeline + metadata table, index
Commit	CAS con trỏ trong catalog	Tạo file log số kế tiếp	Theo timeline, có cơ chế khóa
Partition	Hidden partitioning, partition evolution	Partition kiểu Hive, Liquid Clustering	Partition kiểu Hive, clustering
Điểm mạnh	Nhiều engine, bảng rất lớn	Tích hợp sâu với Spark và Databricks	Upsert, CDC tần suất cao

Bảng 5: So sánh ba table format phổ biến.

Khoảng cách tính năng giữa các format đang thu hẹp: Iceberg v3 có deletion vector giống Delta, Delta UniForm sinh thêm metadata Iceberg để client Iceberg đọc được bảng Delta, còn Apache XTable chuyển đổi metadata qua lại giữa cả ba format [18]. Vì vậy việc chọn format giờ phụ thuộc chủ yếu vào hệ sinh thái engine và catalog mà tổ chức đang dùng.

VI.8. Catalog: lớp điều khiển của Lakehouse

Catalog đảm nhận bốn vai trò: giữ con trỏ metadata và thực hiện CAS, quản lý namespace (`database.table`), phân quyền, và là nơi để engine tìm ra bảng. Nếu catalog ngừng hoạt động, không engine nào đọc hay ghi được bảng. Vì thế catalog là hạ tầng cấp một, cần được vận hành với độ sẵn sàng cao như database chính.

Iceberg REST Catalog [19] chuẩn hóa giao tiếp giữa engine và catalog qua HTTP. Trước đó, mỗi engine cần một connector riêng cho từng loại catalog (Hive Metastore, AWS Glue, JDBC). Giao thức REST còn mở đường cho credential vending: thay vì cấp cho engine khóa truy cập dài hạn vào cả bucket, catalog chỉ cấp token ngắn hạn cho đúng prefix của bảng được phép đọc.

Catalog	Đặc điểm
Apache Polaris	Snowflake khởi tạo năm 2024, trở thành Top-Level Project của Apache đầu năm 2026 [20]; triển khai đầy đủ REST, RBAC, credential vending
Unity Catalog	Trung tâm của nền tảng Databricks; bản mã nguồn mở có ít tính năng hơn bản managed [21]
Lakekeeper	Iceberg REST catalog viết bằng Rust, lưu trạng thái trong PostgreSQL, phù hợp tự triển khai [22]
Project Nessie	Branch và commit kiểu Git ở mức catalog, hỗ trợ giao dịch trên nhiều bảng [23]
Hive Metastore, AWS Glue	Thế hệ catalog trước, vẫn phổ biến trong các hệ thống hiện có
SQL catalog (SQLite, PostgreSQL)	Lưu con trỏ trong một bảng SQL; phần thực hành dùng SQLite, chỉ hợp cho học tập và thử nghiệm

Bảng 6: Một số catalog cho Iceberg và Delta Lake.

Lưu ý

REST giúp di chuyển metadata của bảng giữa các catalog mà không phải chép file Parquet. Tuy vậy, chính sách phân quyền (role, row filter, column mask) không chuyển theo, vì mỗi catalog dùng một mô hình quyền riêng. Chọn catalog là một quyết định dài hạn.

VI.9. Bảo trì bảng (Day-2 operations)

Với Data Warehouse, vendor lo việc bảo trì. Với Lakehouse, đội dữ liệu phải tự làm bốn việc:

1. **Compaction**: gộp file nhỏ do streaming hoặc micro-batch sinh ra.
2. **Expire snapshots**: xóa snapshot quá hạn lưu giữ. Đổi lại, không thể time travel về các snapshot đó nữa.

3. **Xóa orphan file**: dọn file không còn được tham chiếu, sinh ra từ job chết giữa chừng hoặc snapshot đã bị expire.

4. **Gọn metadata**: giới hạn số phiên bản `metadata.json` cũ được giữ lại qua thuộc tính `write.metadata.previous-versions-max`, và gộp các manifest nhỏ bằng `rewrite_manifests`.

Script `04_table_maintenance.py` ghi 300 micro-batch, mỗi batch 1.000 dòng, rồi lần lượt thực hiện các bước trên. Kết quả đo được:

Trạng thái	Snapshot	File cần đọc	Lập kế hoạch	metadata/
Sau 300 micro-batch	300	262	353 ms	901 file, 43,9 MB
Sau compaction	302	1	3 ms	906 file, 44,4 MB
Sau expire snapshots	1	1	2 ms	907 file, 44,4 MB
Sau dọn dẹp metadata, orphan	1	1	3 ms	8 file, 1,0 MB

Bảng 7: Ảnh hưởng của small files và hiệu quả bảo trì trên bảng 300.000 dòng. Thời gian đo trên laptop và thay đổi nhẹ giữa các lần chạy, nên chỉ có ý nghĩa so sánh tương đối.

Compaction giảm thời gian lập kế hoạch hơn 100 lần, nhưng không giảm dung lượng: các file cũ vẫn cần cho time travel. Chỉ sau khi expire snapshot và xóa orphan file, dung lượng mới được thu hồi. Hành vi này phụ thuộc công cụ: `expire_snapshots` của PyIceberg 0.12 chỉ cập nhật metadata, còn procedure cùng tên của Spark xóa luôn các file chỉ thuộc snapshot đã expire. Trên môi trường production, các việc này thường chạy định kỳ bằng các procedure của Spark như `rewrite_data_files`, `expire_snapshots`, `remove_orphan_files`, hoặc giao cho catalog tự động hóa.

VII. So sánh và kết hợp ba kiến trúc

VII.1. Bảng so sánh tổng hợp

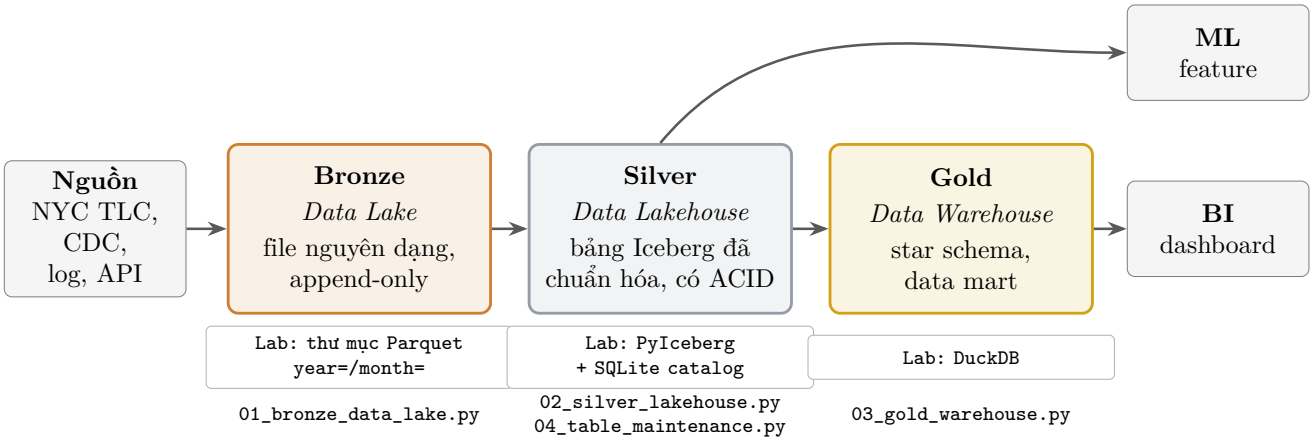
Tiêu chí	Data Warehouse	Data Lake	Data Lakehouse
Sở hữu bảng	Vendor sở hữu cả ba lớp	Không ai	Table format mở + catalog
Storage	Do vendor quản lý, ẩn với người dùng	Object storage, HDFS	Object storage, chủ yếu Parquet
Schema	Schema-on-write	Schema-on-read	Áp khi ghi, tiến hóa qua metadata
Loại dữ liệu	Có cấu trúc	Mọi loại, kể cả ảnh, âm thanh	Có cấu trúc và bán cấu trúc
ACID	Có sẵn	Không	Có, nhờ commit nguyên tử
UPDATE / DELETE	SQL thông thường	Ghi lại cả file hoặc partition	CoW hoặc MoR
Time travel	Có, giới hạn theo gói dịch vụ	Không	Có, theo snapshot
Engine truy cập	Engine của vendor	Mọi engine, qua file	Nhiều engine, qua catalog
Vận hành	Vendor tự động hóa	Script thủ công	Đội dữ liệu tự bảo trì bảng

Bảng 8: So sánh Data Warehouse, Data Lake và Data Lakehouse.

VII.2. Kiến trúc lai trong thực tế

Một sai lầm phổ biến là nghĩ phải chọn duy nhất một trong ba kiến trúc. Nền tảng thực tế thường dùng cả ba, mỗi kiến trúc đảm nhận một vùng trong vòng đời dữ liệu (Hình 7):

- **Vùng thô là Data Lake**: dữ liệu CDC, file từ đối tác, log sự kiện được giữ nguyên dạng. Vì chưa bị biến đổi, vùng này là kho lưu trữ gốc để chạy lại pipeline khi phát hiện lỗi logic ở phía sau.
- **Vùng lõi là Data Lakehouse**: Spark, Flink hay PyIceberg làm sạch dữ liệu thô thành bảng Iceberg hoặc Delta. Đây là bản dữ liệu duy nhất được dùng chung cho ETL, SQL ad-hoc và pipeline feature của ML.
- **Vùng phục vụ là Data Warehouse**: dashboard, báo cáo cần độ trễ thấp và nhiều người dùng đồng thời. Dữ liệu Gold có thể được nạp vào warehouse, hoặc warehouse đọc thẳng bảng Iceberg mà không cần chép.



Hình 7: Kiến trúc lai theo mô hình Medallion. Dòng dưới cùng là công cụ và script tương ứng trong phần thực hành.

VII.3. Medallion và mô hình hóa dữ liệu

Table format bảo đảm bảng `orders` được commit nguyên tử, nhưng không định nghĩa thế nào là "doanh thu thuần" hay "khách hàng đang hoạt động". Medallion architecture tổ chức dữ liệu theo mức độ tinh chỉnh qua ba lớp Bronze, Silver, Gold [24]. Đây là cách tổ chức logic chứ không thay thế mô hình hóa dữ liệu. Mỗi lớp nên dùng phương pháp mô hình hóa phù hợp với vai trò của nó (Bảng 9).

Lớp	Vai trò	Mô hình hóa	Thao tác chính
Bronze	Lưu trữ gốc, không sửa	Giữ nguyên schema nguồn, thêm cột như `_ingested_at`, `_source`	Chỉ append, không lọc theo nghiệp vụ
Silver	Thực thể nghiệp vụ đã làm sạch	3NF kiểu Inmon hoặc Data Vault 2.0	Chuẩn hóa schema, ép kiểu, khử trùng lặp, SCD Type 2
Gold	Dữ liệu sẵn sàng sử dụng	Star schema kiểu Kimball, hoặc One Big Table (OBT)	Tính metric, tổng hợp, mô tả ý nghĩa cột cho BI và AI

Bảng 9: Ánh xạ lớp Medallion với phương pháp mô hình hóa.

Ở lớp Gold, star schema phù hợp khi người dùng chính là nhà phân tích và công cụ BI. One Big Table (một bảng rộng đã `JOIN` sẵn mọi dimension) phù hợp khi cần quét nhanh một bảng duy nhất, ví dụ để tạo feature cho ML.

VII.4. Khi nào chọn kiến trúc nào

Gợi ý lựa chọn

- **Chỉ cần Data Warehouse** khi người dùng chủ yếu là nhà phân tích SQL, dữ liệu có cấu trúc và đội ngũ nhỏ, muốn ít việc vận hành.
- **Luôn giữ một vùng Data Lake** để lưu dữ liệu thô, đa dạng và đến nhanh trước khi xử lý.
- **Dùng Data Lakehouse** khi phải phục vụ cùng lúc ETL theo lô, streaming, SQL ad-hoc và ML trên một bản dữ liệu duy nhất mà không muốn trả giá cho kiến trúc hai tầng.
- **Chọn table format theo hệ sinh thái**: Delta Lake nếu đã gắn chặt với Databricks; Iceberg cùng một REST catalog mở như Polaris hay Lakekeeper nếu cần nhiều engine (Spark, Trino, Flink, DuckDB) hoặc SDK ngoài JVM như PyIceberg.

VIII. Thực hành: NYC Yellow Taxi từ Bronze đến Gold

VIII.1. Môi trường và dữ liệu

Phần thực hành chạy hoàn toàn trên laptop, không cần Docker hay tài khoản cloud. Kiến trúc tổng thể là Hình 7. Bảng 10 liệt kê công cụ cho từng lớp.

Lớp	Kiến trúc	Công cụ	Script
Bronze	Data Lake	Thư mục Parquet, PyArrow, DuckDB	`01_bronze_data_lake.py`
Silver	Data Lakehouse	PyIceberg 0.12, SQLite catalog	`02_silver_lakehouse.py`, `04_table_maintenance.py`
Gold	Data Warehouse	DuckDB 1.5 + extension `iceberg`	`03_gold_warehouse.py`

Bảng 10: Công cụ và script của phần thực hành.

Các API của PyIceberg dùng trong bài được mô tả trong tài liệu chính thức [25].

Dữ liệu là chuyến taxi vàng tháng 1 và tháng 2 năm 2023 do NYC TLC công bố [1], mỗi tháng một file Parquet khoảng 48 MB và khoảng 3 triệu dòng. Script tự tải dữ liệu ở lần chạy đầu. Các lệnh cài đặt (Python 3.10 trở lên):

Cài đặt và chạy

```
cd solution/code
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

```

```
python 01_bronze_data_lake.py      # Bronze: Data Lake và các lỗi data swamp
python 02_silver_lakehouse.py      # Silver: bảng Iceberg, evolution, time travel
python 03_gold_warehouse.py        # Gold: star schema và data mart trên DuckDB
python 04_table_maintenance.py     # Mở rộng: small files và bảo trì bảng
```

Output của lớp Bronze đã được trình bày trong mục IV., nên phần dưới chỉ tập trung vào hai bước còn lại.

VIII.2. Silver: chuẩn hóa schema và nạp vào Iceberg

Bước quan trọng nhất của Silver là đưa dữ liệu các tháng về **một schema đích** trước khi ghi. Hàm `conform` đổi tên cột về chữ thường (xử lý luôn `airport_fee` và `Airport_fee`), ép kiểu theo `SILVER_SCHEMA`, rồi áp bốn quy tắc chất lượng dữ liệu:

02_silver_lakehouse.py: hàm conform

```python
def conform(ym: str) -> pa.Table:
    raw = pq.read_table(bronze_file(ym))
    # Tháng 01 có cột airport_fee, tháng 02 lại là Airport_fee
    raw = raw.rename_columns([RENAME.get(c.lower(), c.lower())
                              for c in raw.column_names])
    tbl = raw.select(SILVER_SCHEMA.names).cast(SILVER_SCHEMA)

    year, month = map(int, ym.split("-"))
    next_month = datetime(year + month // 12, month % 12 + 1, 1)
    start = pa.scalar(datetime(year, month, 1), pa.timestamp("us"))
    end = pa.scalar(next_month, pa.timestamp("us"))
    rules = {
        "pickup ngoài tháng": pc.or_(pc.less(tbl["pickup_at"], start),
                                     pc.greater_equal(tbl["pickup_at"], end)),
        "trip_distance <= 0": pc.less_equal(tbl["trip_distance"], 0),
        "fare_amount <= 0": pc.less_equal(tbl["fare_amount"], 0),
        "dropoff trước pickup": pc.less(tbl["dropoff_at"], tbl["pickup_at"]),
    }
    bad = pc.or_(pc.or_(rules["pickup ngoài tháng"], rules["trip_distance <= 0"]),
                 pc.or_(rules["fare_amount <= 0"], rules["dropoff trước pickup"]))
    return tbl.filter(pc.invert(bad))
```

Output

```
2023-01: Bronze 3,066,766 dòng -> Silver 2,998,337 dòng
    loại vì pickup ngoài tháng    :      48
    loại vì trip_distance <= 0    :  45,862
    loại vì fare_amount <= 0      :  26,159
    loại vì dropoff trước pickup  :       3
```

Một dòng có thể vi phạm nhiều quy tắc cùng lúc, nên tổng các con số bên trên lớn hơn số dòng thực sự bị loại. Tạo bảng và ghi dữ liệu chỉ cần hai lệnh; catalog SQLite ghi lại con trỏ tới `metadata.json`:

02_silver_lakehouse.py: tạo bảng và commit snapshot đầu tiên

```
catalog = load_lab_catalog()       # SqlCatalog: SQLite giữ con trỏ, file nằm trên đĩa
catalog.create_namespace_if_not_exists("silver")
tbl = catalog.create_table("silver.yellow_trips", schema=SILVER_SCHEMA)
tbl.append(conform("2023-01"))     # snapshot 1
```

Khi thử ghi một batch có `vendor_id` kiểu chuỗi, PyIceberg từ chối ngay từ phía client với lỗi `Mismatch in fields`, và snapshot hiện hành giữ nguyên. Cùng một batch sai đó, ở lớp Bronze đã làm hỏng mọi truy vấn trên prefix.

VIII.3. Gold: Star Schema với DuckDB

DuckDB đọc bảng Silver qua `iceberg_scan` trên file `metadata.json` hiện hành, nên chỉ thấy đúng các file thuộc snapshot hiện tại, không bao giờ thấy file rác hay file đã bị xóa. Fact table được tạo như sau:

03_gold_warehouse.py: fact table với grain một chuyến xe

```
CREATE OR REPLACE TABLE fact_trips AS
SELECT
    row_number() OVER (ORDER BY pickup_at)          AS trip_key,
    vendor_id                                       AS vendor_key,
    payment_type                                    AS payment_type_key,
    CAST(strftime(pickup_at, '%Y%m%d') AS INTEGER)  AS pickup_date_key,
    pu_location_id, do_location_id, passenger_count, trip_distance,
    date_diff('minute', pickup_at, dropoff_at)      AS duration_min,
    fare_amount, tip_amount, total_amount,
    coalesce(tip_per_mile, tip_amount / trip_distance) AS tip_per_mile
FROM iceberg_scan('<đường dẫn metadata.json hiện hành>');
```

Biểu thức `coalesce` xử lý hệ quả của schema evolution: file tháng 1 không có `tip_per_mile` nên được tính lại tại chỗ. Trước khi tạo data mart, script chạy các kiểm tra chất lượng dữ liệu:

Output: kiểm tra chất lượng dữ liệu

```
  [PASS] Số dòng fact = số dòng Silver
  [PASS] Không có vendor_key mồ côi
  [PASS] Không có payment_type_key mồ côi
  [PASS] Không có pickup_date_key mồ côi
  [PASS] trip_key là duy nhất

Nếu dim_payment_type thiếu mã 0, INNER JOIN sẽ âm thầm làm mất 135,947 chuyến.
```

Lưu ý

Nhiều tài liệu cũ chỉ liệt kê mã thanh toán từ 1 đến 6. Data dictionary năm 2025 của NYC TLC bổ sung mã 0 là *Flex Fare trip* [9], và dữ liệu hai tháng đầu năm 2023 có 135.947 chuyến mang mã này. Dimension thiếu mã 0 khiến `INNER JOIN` lặng lẽ loại các chuyến đó, và doanh thu trên dashboard bị thấp hơn thực tế mà không có lỗi nào được báo. Kiểm tra khóa ngoại mồ côi phát hiện được lỗi này.

Data mart cuối cùng tổng hợp doanh thu theo ngày và theo hãng, chỉ tính các hình thức thanh toán có phát sinh doanh thu:

Output: 4 dòng đầu của mart_daily_vendor_revenue

```
+------------+----------------------+-------+---------------+------------------+
| full_date  |        vendor        | trips | gross_revenue | avg_duration_min |
+------------+----------------------+-------+---------------+------------------+
| 2023-01-01 | Creative Mobile Tech | 17915 |      505093.0 |             14.9 |
| 2023-01-01 | Curb Mobility, LLC   | 55744 |    1778793.76 |             18.4 |
| 2023-01-02 | Creative Mobile Tech | 16569 |     480764.08 |             14.3 |
| 2023-01-02 | Curb Mobility, LLC   | 46662 |    1509729.55 |             17.1 |
+------------+----------------------+-------+---------------+------------------+
```

VIII.4. Hướng mở rộng

Bộ code dùng ổ đĩa cục bộ và catalog SQLite để chạy được ở mọi máy. Để tiến gần môi trường production, bạn có thể lần lượt:

1. Thay thư mục cục bộ bằng **MinIO** (object storage tương thích S3) chạy bằng Docker, rồi cấu hình PyIceberg ghi lên bucket.
2. Thay SQLite bằng một **Iceberg REST catalog** như Lakekeeper hoặc Apache Polaris; khi đó `load_catalog` nhận `type="rest"` cùng địa chỉ endpoint của catalog.
3. Cho **Trino** hoặc **Spark** cùng đọc bảng mà PyIceberg đã ghi để kiểm chứng tính đa engine. DuckDB cũng ghi được vào bảng Iceberg khi được `ATTACH` tới một REST catalog; `iceberg_scan` trên file metadata như trong bài chỉ cho phép đọc [26].

IX. Lộ trình học tiếp

Để đi từ mức hiểu khái niệm tới mức vận hành được hệ thống thật, có thể học theo năm giai đoạn trong khoảng 16 tuần (Bảng 11), đi từ lớp lưu trữ vật lý lên tới kiến trúc lai hoàn chỉnh.

Giai đoạn	Kiến thức trọng tâm	Công cụ	Sản phẩm kiểm chứng
1. Data Warehouse (tuần 1–3)	Row và columnar, Parquet, schema-on-write, star schema, SCD Type 2	DuckDB, dbt, PyArrow	Star schema trong DuckDB + dbt, có test và SCD2
2. Data Lake (tuần 4–6)	Object storage, schema-on-read, partition kiểu Hive, small files	MinIO, PySpark	Tái hiện partial write và schema drift trên MinIO
3. Lakehouse (tuần 7–10)	Cây metadata, CAS commit, CoW và MoR, hidden partitioning, time travel	PyIceberg, Spark, Delta Lake	Pipeline Silver có schema evolution, partition evolution, time travel
4. Catalog (tuần 11–13)	REST catalog, credential vending, RBAC, đa engine	Polaris hoặc Lakekeeper, Trino, DuckDB	PyIceberg ghi, Trino và DuckDB cùng đọc qua REST catalog
5. Kiến trúc lai (tuần 14–16)	Hợp đồng Bronze/Silver/Gold, compaction, expire snapshots, orphan files	MinIO, Polaris, Spark, DuckDB, dbt	Nền tảng Medallion có lịch bảo trì và số đo trước/sau

Bảng 11: Lộ trình 16 tuần sau bài học.

X. Tổng kết

- Data Warehouse, Data Lake và Data Lakehouse khác nhau ở chỗ ai sở hữu khái niệm "bảng": vendor, không ai, hay một table format mở cùng catalog.
- Data Warehouse cho SQL nhanh, ACID và vận hành tự động, đổi lại chi phí cao và dữ liệu bị khóa trong định dạng riêng.
- Data Lake rẻ và linh hoạt, nhưng thiếu schema enforcement, commit nguyên tử và DELETE, nên dễ thành data swamp. Ngay dữ liệu NYC TLC cũng đổi schema giữa hai tháng liên tiếp.
- Lakehouse giữ file Parquet trên object storage và thêm cây metadata cùng commit bằng CAS, nhờ đó có ACID, schema evolution, partition evolution và time travel.
- Catalog là lớp điều khiển của Lakehouse, còn việc bảo trì bảng (compaction, expire snapshots, dọn orphan file) chuyển sang trách nhiệm của đội dữ liệu.
- Nền tảng thực tế kết hợp cả ba kiến trúc theo Medallion: Data Lake cho Bronze, Lakehouse cho Silver, Warehouse cho Gold. Mô hình hóa dữ liệu (3NF, Data Vault, star schema) vẫn là việc riêng của từng lớp.

Bảng thuật ngữ

Thuật ngữ	Tạm dịch	Mô tả
OLTP	Xử lý giao dịch trực tuyến	Workload gồm nhiều giao dịch nhỏ, mỗi giao dịch đọc hoặc ghi vài dòng, ví dụ tạo đơn hàng hay cập nhật trạng thái chuyến xe.
OLAP	Xử lý phân tích trực tuyến	Workload truy vấn quét nhiều dòng nhưng ít cột, tổng hợp theo nhiều chiều.
Schema-on-write	Áp schema khi ghi	Dữ liệu phải khớp schema đã khai báo trước khi được nạp; dòng sai bị từ chối.
Schema-on-read	Áp schema khi đọc	Dữ liệu được lưu nguyên dạng; engine tự suy ra cấu trúc lúc truy vấn.
Object storage	Lưu trữ đối tượng	Kho lưu file dạng key–value như S3, GCS, ADLS, MinIO; không có thư mục thật, chỉ có prefix của key.
Data swamp	Đầm lầy dữ liệu	Data Lake mất kiểm soát: không ai biết file nào hợp lệ, schema nào là hiện hành.
Open table format	Định dạng bảng mở	Đặc tả metadata mô tả một bảng gồm những file nào, schema, partition và lịch sử phiên bản (Iceberg, Delta Lake, Hudi).
Catalog	Danh mục bảng	Dịch vụ giữ con trỏ tới metadata hiện hành của từng bảng và thực hiện commit nguyên tử.
Snapshot	Ảnh chụp trạng thái	Trạng thái đầy đủ của bảng sau một lần commit.
Manifest	Tệp kê khai	File Avro liệt kê data file kèm thống kê từng cột, dùng để loại file khi lập kế hoạch truy vấn.
Compare-and-swap	So sánh rồi hoán đổi	Phép cập nhật nguyên tử: chỉ đổi con trỏ nếu giá trị hiện tại vẫn đúng như writer đã đọc.
Copy-on-Write	Sao chép khi ghi	Sửa hoặc xóa dòng bằng cách ghi lại toàn bộ file chứa dòng đó.
Merge-on-Read	Gộp khi đọc	Ghi riêng thông tin xóa hoặc sửa và gộp vào dữ liệu gốc lúc đọc.

Thuật ngữ	Tạm dịch	Mô tả
Deletion vector	Vector đánh dấu xóa	Bitmap cho biết những dòng nào trong một data file đã bị xóa.
Time travel	Truy vấn quá khứ	Đọc bảng tại một snapshot hoặc một thời điểm trước đây.
Hidden partitioning	Partition ẩn	Partition được tính từ cột nguồn qua transform như day, month, bucket; người dùng chỉ lọc trên cột gốc.
Compaction	Gộp file	Ghi lại nhiều file nhỏ thành ít file lớn để giảm chi phí đọc và lập kế hoạch truy vấn.
Orphan file	File mồ côi	File nằm trong thư mục bảng nhưng không còn metadata nào tham chiếu tới.
Credential vending	Cấp quyền tạm thời	Catalog cấp token lưu trữ ngắn hạn, chỉ có hiệu lực với đúng bảng mà truy vấn được phép đọc.
Medallion architecture	Kiến trúc huy chương	Tổ chức dữ liệu thành các lớp Bronze, Silver, Gold theo mức độ làm sạch và tinh chỉnh.
Grain	Độ chi tiết	Ý nghĩa của một dòng trong fact table, ví dụ một dòng là một chuyến xe.
Surrogate key	Khóa thay thế	Khóa do warehouse tự sinh, độc lập với khóa của hệ thống nguồn.
SCD Type 2	Dimension thay đổi chậm loại 2	Lưu lịch sử thay đổi thuộc tính bằng cách thêm dòng mới kèm khoảng thời gian hiệu lực.

XI. Tài liệu tham khảo

[1] NYC Taxi and Limousine Commission. “TLC Trip Record Data”, Accessed: Oct. 15, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[2] B. A. Devlin et al., “An Architecture for a Business and Information System”, *IBM Systems Journal*, vol. 27, no. 1, pp. 60–80, 1988.

[3] W. H. Inmon, *Building the Data Warehouse*, 4th ed. Wiley, 2005.

[4] R. Kimball et al., *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.

[5] J. Dixon. "Pentaho, Hadoop, and Data Lakes", Accessed: Oct. 15, 2026. [Online]. Available: https://jamesdixon.wordpress.com/2010/10/14/pentaho-hadoop-and-data-lakes/

[6] M. Armbrust et al., "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics", in *Conference on Innovative Data Systems Research (CIDR)*, 2021.

[7] Apache Software Foundation. "Apache Parquet: File Format", Accessed: Oct. 15, 2026. [Online]. Available: https://parquet.apache.org/docs/file-format/

[8] D. Linstedt et al., *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann, 2015.

[9] NYC Taxi and Limousine Commission. "Data Dictionary – Yellow Taxi Trip Records", Accessed: Oct. 15, 2026. [Online]. Available: https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

[10] Amazon Web Services. "Amazon S3 Update – Strong Read-After-Write Consistency", Accessed: Oct. 15, 2026. [Online]. Available: https://aws.amazon.com/blogs/aws/amazon-s3-update-strong-read-after-write-consistency/

[11] M. Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores", *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.

[12] Apache Software Foundation. "Apache Iceberg Table Spec", Accessed: Oct. 15, 2026. [Online]. Available: https://iceberg.apache.org/spec/

[13] Apache Software Foundation. "Apache Iceberg Releases", Accessed: Oct. 15, 2026. [Online]. Available: https://iceberg.apache.org/releases/

[14] Dremio. "What's New in Apache Iceberg 1.11.0", Accessed: Oct. 15, 2026. [Online]. Available: https://www.dremio.com/blog/whats-new-in-apache-iceberg-1-11-0/

[15] Delta Lake Project. "Delta Transaction Log Protocol", Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/delta-io/delta/blob/master/PROTOCOL.md

[16] Delta Lake Project. "Delta Lake 4.1.0 Released", Accessed: Oct. 15, 2026. [Online]. Available: https://delta.io/blog/2026-03-01-delta-lake-4-1-0-released

[17] Apache Software Foundation. "Apache Hudi Documentation: Timeline", Accessed: Oct. 15, 2026. [Online]. Available: https://hudi.apache.org/docs/timeline

[18] Apache Software Foundation. "Apache XTable (Incubating)", Accessed: Oct. 15, 2026. [Online]. Available: https://xtable.apache.org/

[19] Apache Software Foundation. "Apache Iceberg REST Catalog OpenAPI Specification", Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/apache/iceberg/blob/main/open-api/rest-catalog-open-api.yaml

[20] Apache Software Foundation. "The Apache Software Foundation Graduates Two Open Source Projects from Incubator", Accessed: Oct. 15, 2026. [Online]. Available: https://news.apache.org/foundation/entry/the-apache-software-foundation-graduates-two-open-source-projects-from-incubator

[21] Unity Catalog Project. "Unity Catalog: Open, Multi-modal Catalog for Data and AI", Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/unitycatalog/unitycatalog

[22] Lakekeeper Project. "Lakekeeper: Apache Iceberg REST Catalog Written in Rust", Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/lakekeeper/lakekeeper

[23] Project Nessie. "Project Nessie: Transactional Catalog for Data Lakes with Git-like Semantics", Accessed: Oct. 15, 2026. [Online]. Available: https://projectnessie.org/

[24] Databricks. "What is a Medallion Architecture?", Accessed: Oct. 15, 2026. [Online]. Available: https://www.databricks.com/glossary/medallion-architecture

[25] Apache Software Foundation. "PyIceberg Documentation", Accessed: Oct. 15, 2026. [Online]. Available: https://py.iceberg.apache.org/

[26] DuckDB Foundation. "DuckDB Iceberg Extension", Accessed: Oct. 15, 2026. [Online]. Available: https://duckdb.org/docs/current/core_extensions/iceberg/overview